

Toolkit for programs in Assembly language in the study course DPI343 Computer Organization and Assembly Language

The study course uses the Microsoft Macro Assembler (MASM) language version for the 32-bit environment for x86 processors, as well as the 16-bit version in real address mode in the emulation environment, which is required in the second part of the course.

In order to successfully complete this study course and complete the tasks provided in it, the student must install the development tools for this study course on his/her home computer with Windows operating system, including Microsoft Visual Studio 2022. All tools used in the course are free of charge and without time limits.

If the student does not have a personal computer that can be used to install and use these tools, then he/she can use the computers in the RTU FCSITE computer classes in accordance with the instructions published on the course page. The use of other software development tools for this study course is not supported.

Where can I get the development tools?

On the course page in the e-study environment, you can find:

- 1) a link to the Microsoft Visual Studio 2022 installation,
- 2) _COAL2025_32.zip – a toolkit for creating programs in a 32-bit environment,
- 3) _COAL2025_16.zip – a toolkit for creating programs in a 16-bit environment.

You must install Microsoft Visual Studio 2022 (Community edition, which is free of charge and without time limits) on your home computer.

You just need to unzip the archived toolkits in a place convenient for work on your computer or in the "cloud". It is recommended to copy them to your RTU account's OneDrive so that you can use them from any computer on which Visual Studio is installed (your own computer or one of the computers in the faculty computer class).

Toolkit for 32-bit environment

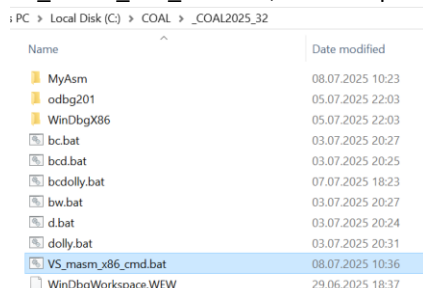
The toolkit includes:

- WinDbgX86 folder with Microsoft debugger WinDbg (from Windows SDK),
- odbg201 folder with debugger ollydbg.exe (if someone finds it more convenient than the professional tool WinDbg),
- MyAsm folder, which contains some examples of assembler programs and where you can store your own program files,
- WinDbgWorkspace.WEW – WinDbg debugger configuration file,
- scripts for more convenient work with development tools:
 - VS_masm_x86_cmd.bat – environment configuration and opening of console window,
 - bc.bat – builds the executable code for console mode (**build for console**),
 - bw.bat – builds the executable code for Windows mode (**build for Windows**),
 - bcd.bat – builds the executable code for console mode and opens in debugger WinDbg (**build for console and debug**),
 - d.bat – opens the executable code in the WinDbg debugger,
 - bcdolly.bat – builds the executable code for console mode and opening it in the ollydbg debugger,
 - dolly.bat – opens the executable code in the ollydbg debugger.

How to create a program?

The program source code must be prepared with a plain text editor (for example, Notepad) and saved in a file with the extension .asm. For example, in the MyAsm folder or some other.

Once the program source code is prepared, you must first open a console window by executing the script VS_masm_x86_cmd.bat, for example by double-clicking on this file:



What does the VS_masm_x86_cmd.bat script do?

This script prepares the development environment and then opens a console window. The script looks for a version of Visual Studio installed on the computer, because the other scripts will use the compiler and linker that are part of Visual Studio. If Visual Studio is installed on the computer and found (it should be installed in the standard way, without changing the name), then it looks for and executes the configuration script vcvars32.bat in the Visual Studio package, which prepares the necessary environment variables and other settings. Then this script looks for the debugger WinDbg either in the Windows SDK already installed on this computer, or in the WinDbg folder in our course toolkit. If the previous steps have been successfully completed, then the script also sets some additional necessary environment variables and starts cmd.exe, which opens a console window. In this window, the script displays a list of subfolders of the current folder and a brief instruction on what to do next.

```
C:\WINDOWS\system32\cmd.exe
*****
** Visual Studio 2022 Developer Command Prompt v17.14.9
** Copyright (c) 2025 Microsoft Corporation
*****
[vcvarsall.bat] Environment initialized for: 'x86'
Debugger windbg.exe (x86) path=C:\Program Files (x86)\Windows Kits\10\Debuggers\
x86

Subfolders at C:\COAL\_COAL2025_32 :
-----
MyAsm
odbg201
WinDbgX86
-----

Enter the command CD to go to the folder with the source code files
C:\COAL\_COAL2025_32>
```

It is recommended that you familiarize yourself with the structure of this and other scripts in the development toolkit by opening them, for example, with Notepad. For additional information on the various options for creating such scripts, see, for example, the Microsoft Learn on-line pages and other Internet sources.

How to compile and run an assembly program?

In the console window, enter the command cd (change directory) to go to the folder where the program source code is stored. It is most convenient if it is a subfolder of the current folder, for example, MyAsm:

```
Enter the command CD to go to the folder with the source code files
C:\COAL\_COAL2025_32>cd myasm
C:\COAL\_COAL2025_32\MyAsm>
```

To build the executable (i.e. compile the source code and create the corresponding .exe file with a linker), you need to run the script bc.bat, and pass the name of the source file without the .asm extension as a parameter (the script will add it automatically).

For example, if the source code is in the file hello.asm, then:

```
C:\COAL\_COAL2025_32\MyAsm>bc hello
Microsoft (R) Macro Assembler Version 14.44.35213.0
Copyright (C) Microsoft Corporation. All rights reserved.

Assembling: hello.asm
Microsoft (R) Incremental Linker Version 14.44.35213.0
Copyright (C) Microsoft Corporation. All rights reserved.

C:\COAL\_COAL2025_32\MyAsm>
```

If there are no compilation errors, then an executable file will be created in the current folder, in this case hello.exe, which can be executed by entering the file name:

```
C:\COAL\_COAL2025_32\MyAsm>hello
Hello!
```

If there are errors in the program, the compiler outputs error messages to the console screen (with the numbers of the erroneous lines indicated), for example, if the call instruction is written incorrectly on line 17 (cal instead of call):

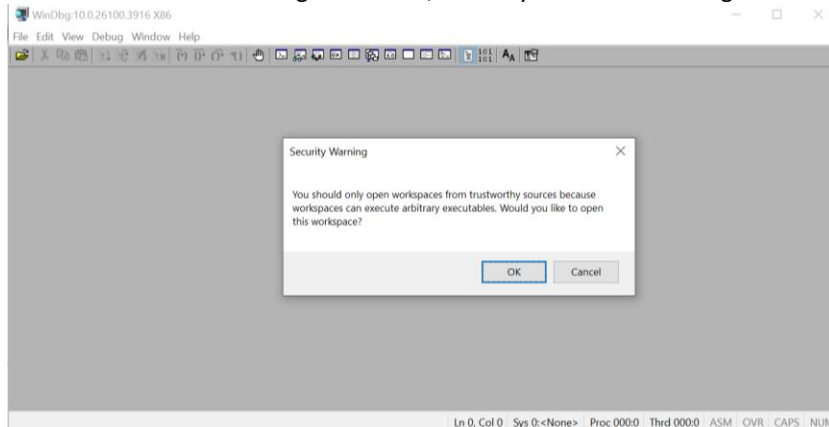
```
C:\COAL\_COAL2025_32\MyAsm>bc hello
Microsoft (R) Macro Assembler Version 14.44.35213.0
Copyright (C) Microsoft Corporation. All rights reserved.

Assembling: hello.asm
hello.asm(17) : error A2008:syntax error : cal
***** Build Error *****
C:\COAL\_COAL2025_32\MyAsm>
```

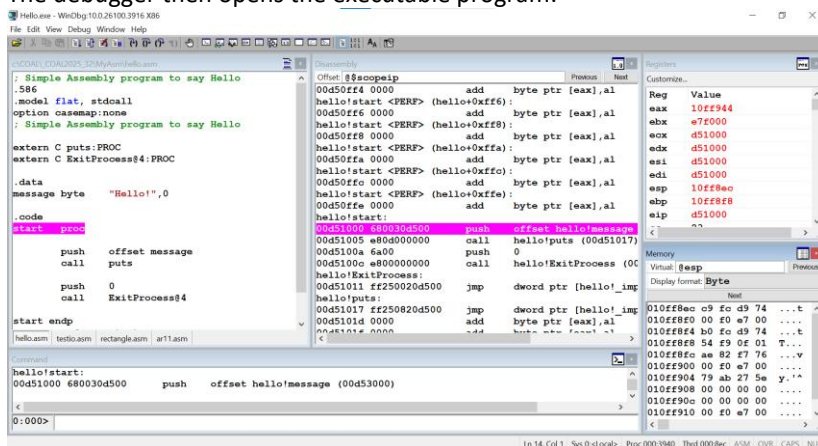
To build the executable code and open it in the debugger, you need to run the script bcd.bat, which works similarly to bc.bat, but opens the executable file (for example, hello.exe) in the WinDbg environment:

```
C:\COAL\_COAL2025_32\MyAsm>bcd hello
```

The debugger will use configuration information (which windows are open, how they are arranged, font size, color usage, etc.) from the WinDbgWorkspace.WEW file, which is included in the toolkit. This file contains a configuration that is convenient for daily work, but you can change it and save the new configuration in this same file (File->Save Workspace to File... menu). It is not recommended to save the configuration with File->Save Workspace, because then it is not written to your own file, but to the Windows Registry. The debugger always asks whether it is allowed to use information from the configuration file, and all you have to do is agree with OK:

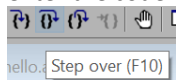


The debugger then opens the executable program:

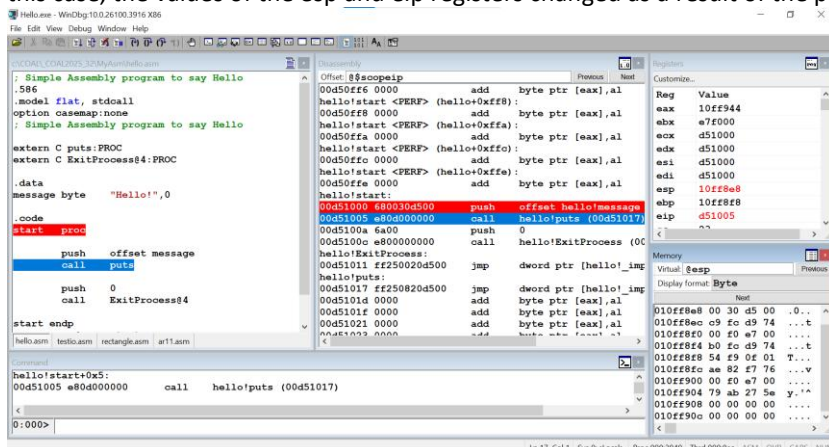


The left-hand window shows the program source file that WinDbg finds automatically. The middle window contains the code generated by the disassembler. On the right-hand side is a window with the current register values in hexadecimal, below is a window with the memory contents. The highlighted line in the disassembler window is the initial stopping point after the executable code has been loaded into memory, and is the first processor instruction of the program to execute (in this case, the push instruction).

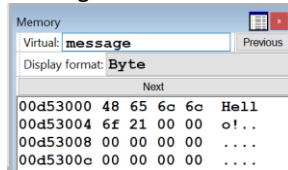
To execute one next instruction, click the "Step over" button, or press key F10. Or "Step into" (F11 or F8), if need to enter the code of the procedure.



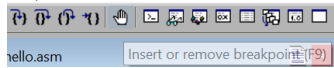
One next instruction will be executed and the new content of the registers will be displayed in the register window (in this case, the values of the esp and eip registers changed as a result of the push instruction):



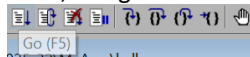
To view the contents of memory at a specific address, enter either the corresponding identifier or the address value as a hexadecimal number in the Memory address input field. For example, to view the contents of a memory field starting at the address message:



To add a breakpoint, place the cursor on the desired line in either the source code window or the disassembler window and then click the "Insert or remove breakpoint" button, or press key F9:



Then, using the "Go" button (or pressing key F5), you can execute the code up to this point:



If you need to re-load executable code and restart debugging from the beginning, you can use the "Restart" button (or press Ctrl+Shift+F5).

You can easily learn the rest of WinDbg's features on your own, if necessary.

You can also learn more on other .bat script operations on your own.

Toolkit for 16-bit Environment

The second part of the course covers interrupt handling and the principles of operation of various computer components.

Programs for experiments with hardware interrupts and access to computer components will be created for the 16-bit environment in real address mode and executed only in the emulation environment.

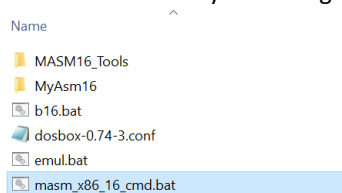
The toolkit for the 16-bit environment includes:

- the MASM16_Tools folder with the Microsoft compiler and link editor for the 16-bit environment, the emulation environment DOSBox tools, as well as the debugger afd.exe,
- the MyAsm16 folder, which contains some examples of assembler programs for the 16-bit environment and where you can store your own developed program files,
- dosbox-0.74-3.conf – a configuration file for the emulation environment,
- scripts for more convenient work with development tools:
 - masm_x86_16_cmd.bat – environment configuration and opening of console window,
 - b16.bat – builds the executable in .exe format,
 - emul.bat – opens the DOSBox emulation environment.

How to create a program?

The program source code must be prepared with a simple plain text editor (for example, Notepad) and saved in a file with the extension .asm. For example, in the folder MyAsm16 or some other.

When the program source code is prepared, you must first open a console window for working with 16-bit environment tools by executing the script masm_x86_16_cmd.bat, for example, by double-clicking on this file:



This script prepares the development environment and then opens a console window. In this window, the script displays a list of subfolders of the current folder and a brief instruction on what to do next.

```
C:\WINDOWS\system32\cmd.exe

Subfolders at c:\COAL\COAL2025_16 :
-----
MASM16_Tools
MyAsm16
-----

Enter the command CD to go to the folder with your 16-bit source code files
Use b16.bat to build .exe executable (example: b16 Hello)
Run emul.bat to open emulation environment for 16-bit apps in the current folder
In the emulation environment window run your 16-bit apps
and use afd.exe for debugging (example: afd hello.exe)

c:\COAL\COAL2025_16>
```

For example, to build the executable code for the Hello.asm program, you need to go to the MyAsm16 folder and run the script b16.bat, specifying the name of the program's source code file as a parameter. The source code is compiled and, if there are no errors, the executable code is built in .exe format.

```
C:\WINDOWS\system32\cmd.exe

C:\COAL\COAL2025_16\MyAsm16>b16 hello
Assembling: hello.asm
-----Build completed successfully-----
C:\COAL\COAL2025_16\MyAsm16>
```

You can then open the emulation environment by executing the emul.bat script:

```
C:\COAL\COAL2025_16\MyAsm16>emul
DOSBox-0.74-3 folder path=C:\COAL\COAL2025_16\MASM16_Tools\DOSBox-0.74-3
C:\COAL\COAL2025_16\MyAsm16>
```

The DOSBox emulation environment window opens:

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip: 0, Program: DOSBOX
Drive W is mounted as local directory c:\COAL\COAL2025_16\MyAsm16\
W:\>
```

The emulation environment window also displays the full path to the folder containing the executable code. It remains to enter the name of the executable program and run this program in the emulation environment:

```
DOSBox 0.74-3, Cpu speed: max 100% cycles, Frameskip: 0, Program: DOSBOX
Drive W is mounted as local directory C:\COAL\COAL2025_16\MyAsm16\
W:\>hello
Hi! This is executable code in .EXE format for 16-bit environment
W:\>
```

The afd.exe debugger can only be opened in the DOSBox window of the emulation environment, passing the name of the executable code file as a parameter. You will be able to easily learn how to work with this debugger on your own.